



UNIVERSITÉ DE
MONTPELLIER



STATISTIQUE
SCIENCE DES DONNÉES BIOSTATS
UNIVERSITÉ DE MONTPELLIER



FACULTÉ DES SCIENCES
DE MONTPELLIER

M2 STATISTIQUES ET SCIENCES DES DONNÉES – BIOSTATISTIQUES –

RAPPORT DE STAGE

Titre du stage

Guillaume BOULAND

Tuteur académique :

Nicolas Meyer

Tuteurs professionnels :

Mai Nguyen-Chi

Benjamin Charlier

Jury :

Ali Gannoun

Élodie Brunel

Nicolas Meyer



2023 – 2024

Remerciements

blabla

Résumé

Résumé

Abstract

Table des matières

1	Introduction	8
	Cadre du stage	8
	Contexte	8
	Question	8
	Objectifs	8
2	Segmentation automatique d'images microscopiques de macrophages	9
2.1	Segmentation d'images	9
2.1.1	<i>Image numérique</i>	9
2.1.2	<i>Différentes approches de segmentation d'images</i>	10
2.1.3	<i>Cartesian Genetic Programming (CGP)</i>	11
2.1.4	<i>Kartezio</i>	13
2.2	Évaluation des performances	14
2.3	Création de modèles de segmentation	16
2.4	Visualisation de la qualité des modèles	19
2.5	Segmentation automatique d'images avec MacTrack2	20
3	Suivi temporel des macrophages et fine-tuning d'un modèle de fondation	22
3.1	Tracking à partir de la segmentation d'images	22
3.1.1	<i>Post-traitement</i>	24
3.1.2	<i>Analyse des signaux calciques</i>	24
3.2	Introduction	24
3.3	Fine-tuning de SAM-2	26
3.3.1	<i>Définition</i>	26
3.3.2	<i>Jeu de données</i>	26
3.3.3	<i>Hyper-paramètres</i>	27
3.3.4	<i>Fonctions de perte</i>	27
3.3.5	<i>Optimisation</i>	29
3.3.6	<i>Résultats et comparaisons</i>	30
3.4	Propagation dans une vidéo	30
4	Conclusion	31
	Disponibilité du code	31
5	Matériel et Méthodes	32

A	Annexes	33
A.1	Superposition des deux canaux	33
A.2	Liste des fonctions de traitement d'images disponibles dans Kartezio	33

Table des figures

2.1	cut tail 3hpa, vois Fig. A.1 pour la superposition des deux canaux .	9
2.2		11
2.3	Exemple de graphe orienté acyclique de notre propre modèle (voir section 2.3).	12
2.4		14
2.5	Schématisation de l'IoU (figure issue de [18]).	16
2.6		17
2.7	Macrophages observés à différents temps post-amputation pour un même poisson.	17
2.8		18
2.9	Exemples de comparaisons entre la vérité annotée (en bleu) et les prédictions du modèle choisi (en rouge) sur des images issues de l'échantillon d'entraînement (à gauche) et de test (à droite).	20
2.10	Illustrations des différentes sorties de la fonction <code>locate</code>	21
3.1		24
3.2	Architecture de SAM-2 (<i>figure issue de [27]</i>).	25
3.3	Image d'origine (à gauche) comprenant des macrophages et leur détourage (à droite).	27
A.1	Superposition de deux canaux de couleurs.	35

Liste des tableaux

2.1	Exemples de performances des modèles entraînés sur les différents jeux de données.	19
3.1	Extrait d'un fichier <code>dataset.csv</code> nécessaire pour la reconnaissance des images et des masques pour Kartezio.	23
3.2	Différents exemples des valeurs que peuvent prendre les sorties du <i>mask decoder</i> (logits), leur interprétation et la valeur de la probabilité associée (par transformation sigmoïde).	29
A.1	Fonctions et descriptions	34

Chapitre 1

Introduction

Cadre du stage

Contexte biologique

mehari . . .
lignée transgénique obtention des images, microscopie

Question

Objectifs

Les macrophages sont des cellules immunitaires jouant un rôle critique dans de nombreuses maladies. Ils peuvent changer d'état suite à une blessure à travers un processus appelé la polarisation. Les signaux de calcium dans les macrophages sont un candidat prometteur qui pourrait permettre d'expliquer ce phénomène. Nous cherchons donc à développer un outil permettant de segmenter automatiquement des macrophages dans des images et vidéos de microscopie afin d'étudier la signature d'un macrophage à partir de l'intensité de calcium dans la cellule.

Nous commencerons par segmenter automatiquement des macrophages dans des images de microscopie. Nous étendrons ensuite cette segmentation au domaine temporel, en réalisant un tracking des macrophages dans des vidéos. Nous quantifierons ensuite les intensités de calcium dans chacun des macrophages segmentés afin d'en déduire leur signature à travers une analyse statistique.

La méthode développée, sous la forme d'une librairie Python disponible à tous nommée MacTrack2, aura pour but d'être intuitive car destinée à être utilisée par des biologistes. Elle contiendra des exemples exécutables en un clic et avec une documentation détaillée sur un site internet.

Chapitre 2

Segmentation automatique d'images microscopiques de macrophages

2.1 Segmentation d'images

2.1.1 Image numérique

Les images que nous traitons sont obtenus par microscopie confocale de zebrafish ayant été génétiquement modifiés par la double lignée transgénique, comme nous l'avons vu dans l'**Introduction**. Cette microscopie permet d'observer deux types de signaux : la présence des macrophages et la présence de calcium dans le cytoplasme de ceux-ci, à l'aide d'une excitation d'un laser à une certaine longueur d'onde. Nous séparons ces deux signaux sur deux canaux. Les images sont donc en niveau de gris où chaque pixel est codé par un nombre entre 0 (noir) et 255 (blanc). Nous ajoutons artificiellement des couleurs aux longueurs d'onde auxquelles les protéines fluorescentes sont activées par le laser du microscope. Ainsi, nous avons un canal rouge pour la présence des macrophages et un canal vert pour la présence de calcium dans le cytoplasme de ces cellules (Fig. 2.1).

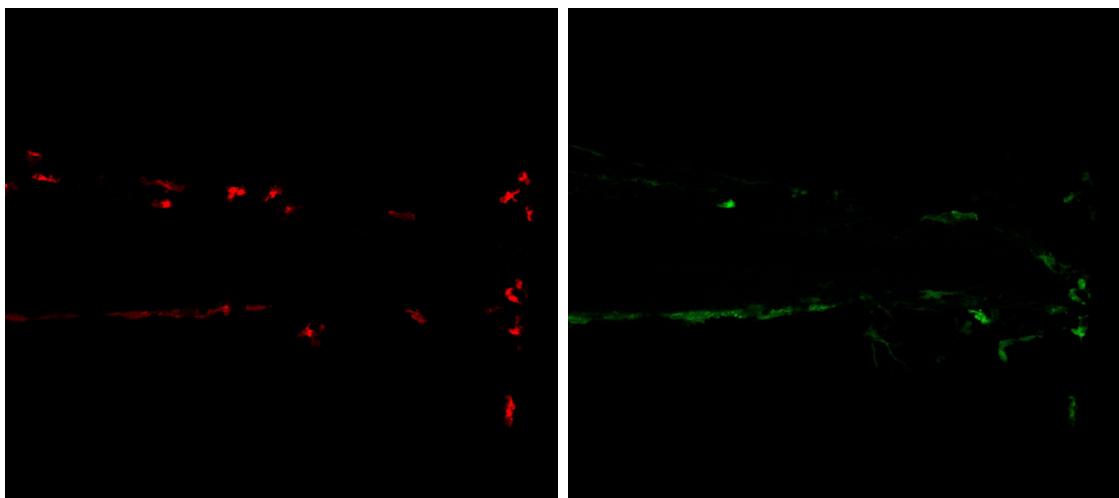


Figure 2.1. cut tail 3hpa, vois Fig. A.1 pour la superposition des deux canaux

2.1.2 Différentes approches de segmentation d'images

La segmentation d'images est un domaine de la vision par ordinateur (*Computer Vision* ou CV) qui consiste à diviser une image en plusieurs régions ou objets d'intérêt. Ici, les objets d'intérêt sont des macrophages, mais la segmentation est un problème plus général, applicable à de nombreux domaines. Les régions ou objets d'intérêts peuvent prendre plusieurs formes comme par exemple des rectangles permettant de délimiter la présence d'un objet. Elle sert à **détecter** tous les objets d'intérêt dans une image et peut même être utilisée pour labelliser ces objets [1, 2, 3, 4].

La **segmentation sémantique** est une autre approche de segmentation d'images où on affecte à chaque pixel une classe d'objet, et ce pour chaque objet de nature (ou sémantique) différente. Elle ne fait donc pas la différence entre deux voitures de couleur ou modèle différents par exemple, mais les classifera chacun comme étant une voiture.

La **segmentation d'instance** est une approche plus complexe qui se concentre sur la distinction des instances, c'est-à-dire des objets individuels, même les instances occluses de la même classe d'objet. Si on reprend notre exemple, les voitures sont maintenant segmentées individuellement, même si elles sont identiques, puisqu'elles sont vues comme des objets différents mais appartenant à la même classe « voiture ».

La **segmentation panoptique** combine les deux précédentes. Elle englobe à la fois la classification sémantique de chacun des pixels de l'image et la délimitation de chaque instance d'objet, mais son coût de calcul est plus élevé.

Dans notre cas, nous cherchons à segmenter des macrophages, donc sur le canal rouge de l'image. Nous avons donc besoin d'une segmentation d'instances car nous voulons détecter chaque macrophage tout en faisant attention à ne pas les confondre entre eux.

Cependant, la segmentation d'instances fait appel à beaucoup de paramètres et des méthodes comme les réseaux de neurones convolutifs. Ces paramètres sont optimisés via une fonction de perte, ce qui nécessite un apprentissage supervisé sur des milliers d'images annotées. Or, annoter à la main un grand nombre d'images demande un temps énorme que nous n'avons pas.

De plus, pour pouvoir segmenter autant d'images, il faut au préalable réaliser toutes les manipulations telles que la reproduction de poissons, le triage des œufs, la sélection des larves, la coupe de la nageoire caudale en fonction de la condition que l'on cherche à observer, l'observation et l'enregistrement microscopique pendant 40 minutes (durée des films que nous avons) à 3, 6, 24, 48 et 72 heures post-amputation, pour chacun des poissons sélectionnés et pour finir le traitement des films sorties de microscopie pour qu'ils soient exécutables. Cela demande un coût humain et financier énorme.

En se tournant vers les modèles de classification sémantique, il avait été découvert l'année précédente *Kartezio* [5], une librairie Python développée en 2023, qui a pour promesse de segmenter des objets dans des images biomédicales en concu-

rençant les modèles basés sur des réseaux de neurones bien connus du domaine comme *Cellpose* [6, 7, 8], *Mask R-CNN* [9] ou encore *Stardist* [10]. Plus précisément, *Kartezio* a pour avantage de générer des pipelines de segmentation robustes et interprétables à partir de peu d'images annotées, en se basant sur la *Cartesian Genetic Programming* (CGP) [11].

2.1.3 Cartesian Genetic Programming (CGP)

La CGP [11], initialement créé pour optimiser des circuits électroniques [12], est une approche de programmation génétique [13] (*Genetic Programming* (GP)) qui utilise des graphes pour coder des programmes informatiques (Fig. 2.2). La GP est un algorithme évolutif, c'est-à-dire qu'il s'inspire les éléments de la biologie évolutive (génotype, génome, phénotype, sélection, recombinaison ou *crossover*, reproduction, mutation, ...) pour résoudre des problèmes d'optimisation discrète complexes. Le terme « cartésien » de la CGP vient du fait que les modèles générés sont représentés grâce à une grille de nœuds, où chaque nœud représente une fonction ou une opération. Chaque nœud est connecté à d'autres nœuds, formant ainsi un graphe orienté acyclique (Fig. 2.3). Le CGP est particulièrement adapté pour les problèmes de classification et de régression, mais il peut également être utilisé pour la segmentation d'images.

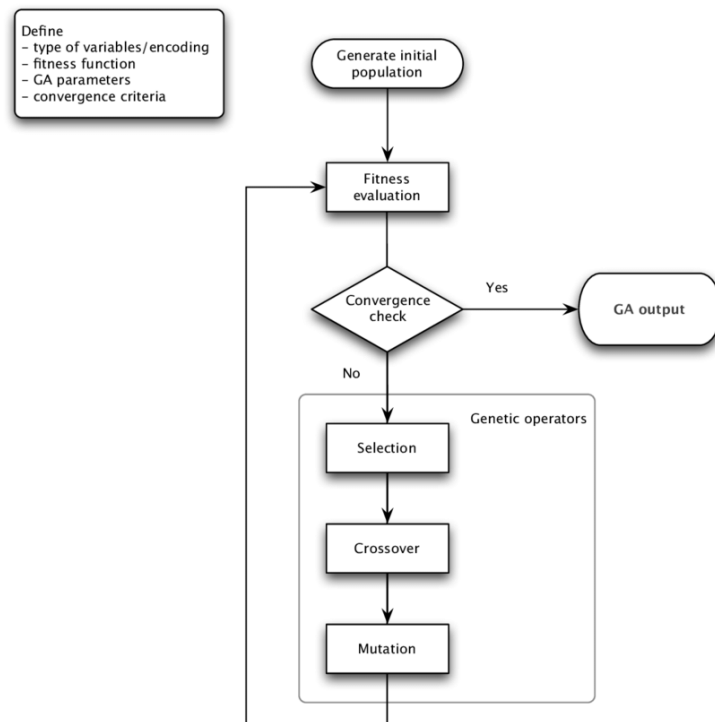


Figure 2.2

Chaque graphe peut être représenté par une chaîne d'entiers, qu'on appelle le génotype. Celui-ci est ensuite décodé pour obtenir le phénotype, qui est le programme exécutable. Le phénotype est obtenu en parcourant le graphe, en commençant par

les nœuds d'entrée (*input*) et en appliquant les fonctions de chaque nœud aux entrées correspondantes.

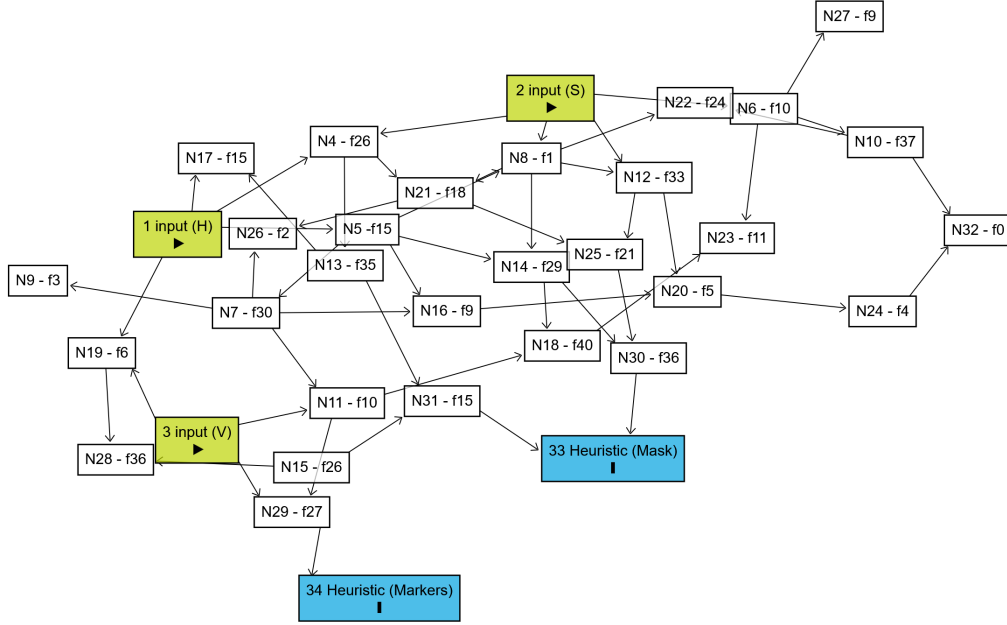


Figure 2.3. Exemple de graphe orienté acyclique de notre propre modèle (voir section 2.3).

Nx avec x le numéro du nœud et fy avec y le numéro de la fonction parmi une liste de fonctions (voir Tab. A.1 pour la numérotation des fonctions). Graphe obtenu avec Dagitty ([14]).

Un modèle CGP utilise une stratégie évolutive (*evolutionary strategy* (ES)) qui va décider de quel opérateur génétique appliquer à chaque génération. Nous avons par exemple la stratégie $(1 + \lambda)$ où un individu (*parent*) génère λ descendants (*offspring*) par mutation et le meilleur de ceux-ci, selon une fonction d'adaptation (*fitness*), est conservé et deviendra l'individu parent de la génération suivante. Si les performances du meilleur descendant sont inférieures à celles du parent, alors la descendante complète est rejetée. On parle de remplacement strict. On peut aussi avoir une stratégie de mutation ciblée, où on altère aléatoirement les gènes (nœuds), en modifiant les fonctions et les connexions, avec un taux de mutation ciblée. Il existe une stratégie de sélection par tournoi, où l'on compare de sous-ensembles d'individus et on sélectionne le plus performant de chacun des groupes, qui est ensuite utilisé pour la reproduction. Il existe des stratégies hybrides qui combinent le CGP avec des méthodes d'apprentissage automatique comme le Deep Learning.

La CGP, contrairement à d'autres approches de GP, n'utilise pas vraiment de recombinaison (*crossover*) et privilégie plutôt des mutations ciblées sur sa population d'individus. De plus, la CGP a une propriété de neutralité génétique, c'est-à-dire que certains changements dans le génotype ne seront pas forcément des changements visibles dans le phénotype. Cependant, ces nœuds servent à explorer l'espace de recherche d'une manière plus efficace. Certains nœuds, d'un autre côté, qui sont eux

efficaces peuvent être facilement dupliqués et ré-utilisés. Cela nous permet d'obtenir des solutions plus compactes en restant tout autant efficaces. Dans le cadre de notre problème, qui est la segmentation d'images, cette particularité du CGP est très utile car souvent en segmentation d'images, la même opération est souvent utilisée plusieurs fois dans différentes parties de l'image.

La stratégie évolutive la plus utilisée est la stratégie $(1 + \lambda)$ car elle favorise la simplicité et la stabilité du processus d'évolution, tout en restant efficace et en limitant la taille de la population. La mutation reste l'opérateur génétique principal utilisé dans cette stratégie. C'est la même stratégie évolutive qu'utilise *Kartezio* pour générer des pipelines de segmentation d'images biomédicales.

2.1.4 Kartezio

Kartezio [5] est une stratégie computationnelle de segmentation d'images biomédicales se basant sur le CGP [11]. Disponible sous la forme d'une librairie Python, cette méthode est capable de générer des pipelines de segmentation d'images qui sont à la fois clairs et interprétables, en assemblant et paramétrisant des fonctions connues de traitement d'images issues de la librairie *OpenCV* [15, 16]. La liste de cette librairie disponible dans *Kartezio* est disponible en Annexe A.2 (Tab. A.1). Les fonctions de cette banque de fonctions de traitement d'images est la même qu'utilise le logiciel ImageJ [17], qui est un logiciel que les biologistes en général et plus particulièrement le laboratoire dans lequel j'étais utilisent pour traiter les images brutes de microscopie qu'ils obtiennent.

Ainsi, l'explicabilité de *Kartezio* couplée à une banque de fonctions déjà connue des praticiens a poussé l'équipe, l'année précédente, à utiliser cette méthode pour segmenter les macrophages sur des images. Sachant que ce projet est destiné à être utilisé ultérieurement par l'équipe, il était nécessaire d'utiliser une méthode rapide, efficace, compréhensible et intelligible pour les biologistes.

Un autre avantage de cette méthode, grâce à son implémentation de la CGP, est que le nombre de données (images) annotées nécessaires pour entraîner un modèle de segmentation est nettement réduit par rapport aux méthodes dites état de l'art que sont par exemple les modèles d'apprentissage profond (*Deep Learning*). *Kartezio* est donc une méthode que l'on appelle *Few-Shot Learning*, c'est-à-dire que l'on a besoin seulement de quelques images annotées pour entraîner un modèle qui ait des performances satisfaisantes. La Fig. 2.4 issue de l'article de Cortacero et al. [5] montre bien la structure de leur modèle, héritée du CGP.

Contrairement au CGP classique, *Kartezio* introduit le concept de nœud non évolutifs (*non evolvable*, voir Fig. 2.4) qui sont le *preprocessing*, qui permet de convertir l'image originale en un format préférentiel, ici le format HSV (*Hue*, *Saturation*, *Value* ou TSV en français pour Teinte, Saturation et Valeur) qui constitue ensuite respectivement les 3 premiers nœuds d'entrée : $\mathcal{N}_1$, $\mathcal{N}_2$ et $\mathcal{N}_3$. Cela permet d'uniformiser les images d'entrées.

Ainsi, nous avons une méthode qui fonctionne pour tous les formats d'images

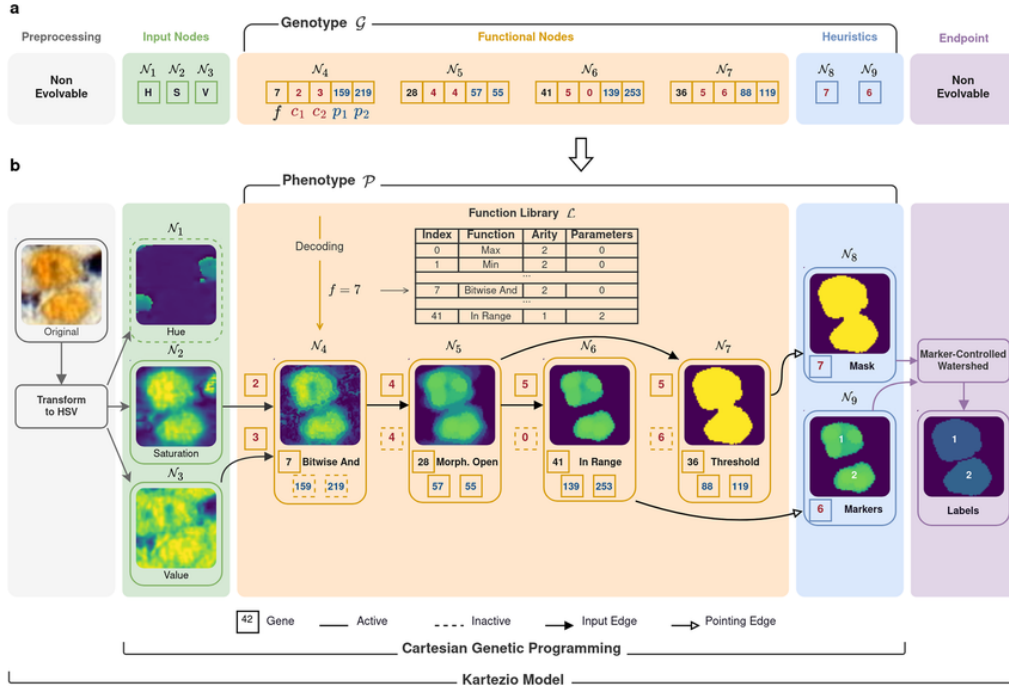


Figure 2.4

classiques (png, jpg, ...) et même les plus flexibles comme le format TIFF (*Tag Image File Format*), qui est un format utilisé notamment en photographie pour le peu de perte de pixels qu'il engendre quand on modifie un tel fichier, mais aussi pour le fait qu'il supporte les images sur plusieurs couches comme les *z-stacks* qui sont beaucoup utilisés en microscopie confocale *in vivo*, comme c'est le cas pour nous.

La dernière opération du pipeline est nommée *Endpoint* et est l'application d'une fonction choisie par l'utilisateur qui permet de créer un arrêt intentionnel des processus d'optimisation pour que celui des nœuds d'heuristique prenne place. Il fait partie intégrante du processus évolutif du génotype puisqu'il agit comme un point d'arrêt où à chaque génération on vérifie qu'il n'est pas dépassé (si c'est un seuil par exemple), mais il n'est pas voué à évoluer.

2.2 Évaluation des performances

Dans toute méthode de segmentation d'images, il est nécessaire d'avoir une métrique qui va nous permettre d'évaluer la qualité de l'apprentissage. La métrique la plus utilisée est l'**IoU** (*Intersection over Union*). On peut la définir comme étant le rapport entre l'intersection (en tant qu'aire) de la prédiction du modèle et de la

vérité annotée et de leur union, pour un objet dans une image (Fig. 2.5) :

$$\text{IoU}(M_k, G_k) = \frac{|M_k \cap G_k|}{|M_k \cup G_k|} = \frac{\sum_{i,j} M_k[i, j] \cdot G_k[i, j]}{\sum_{i,j} M_k[i, j] + \sum_{i,j} G_k[i, j] - \sum_{i,j} M_k[i, j] \cdot G_k[i, j]}$$

avec :

- $i \in \{1, \dots, H\}$ et $j \in \{1, \dots, W\}$ la hauteur et la largeur respectivement de l'image,
- $M_k = (M_k[i, j])_{i,j}$ le masque prédit pour l'objet k , une matrice binaire de taille $H \times W$ où chaque pixel vaut 1 s'il est dans le masque prédit, c'est-à-dire que si la probabilité que le pixel (i, j) appartienne au masque de l'objet k est supérieure à 0.5, sinon il vaut 0. On a donc :

$$M_k[i, j] = \begin{cases} 1 & \text{si } p > 0.5, \\ 0 & \text{sinon.} \end{cases}$$

avec p la probabilité que le pixel (i, j) appartienne au masque de l'objet k ,

- $G_k = (G_k[i, j])_{i,j}$ le masque de vérité pour l'objet k , une matrice binaire de taille $H \times W$ où chaque pixel (élément de la matrice) vaut 1 s'il est dans le masque de vérité, sinon il vaut 0,
- $|M_k \cap G_k| = \sum_{i,j} M_k[i, j] \cdot G_k[i, j]$ l'intersection entre le masque prédit et le masque de vérité, autrement dit le nombre de pixels en commun entre les deux masques,
- $|M_k \cup G_k| = \sum_{i,j} M_k[i, j] + \sum_{i,j} G_k[i, j] - \sum_{i,j} M_k[i, j] \cdot G_k[i, j]$ l'union entre le masque prédit et le masque de vérité, autrement dit le nombre de pixels dans au moins un des deux masques,
- $\sum_{i,j} M_k[i, j]$ le nombre de pixels du masque prédit,
- $\sum_{i,j} G_k[i, j]$ le nombre de pixels du masque de vérité.

Pour obtenir la valeur de l'IoU pour une image entière, on fait la moyenne des IoUs de tous les objets k segmentés dans l'image :

$$\text{mIoU}(M, G) = \frac{1}{K} \sum_{k=1}^K \text{IoU}(M_k, G_k) \quad (2.1)$$

où K est le nombre total d'objet dans une image, $M = (M[i, j])_{i,j}$ le masque prédit pour l'image (sous la forme d'une matrice binaire de taille $H \times W$) et $G = (G[i, j])_{i,j}$ le masque de vérité pour l'image (sous la même forme que M).

Il s'agit d'une valeur comprise entre 0 et 1, où 0 signifie que la prédiction ne recouvre pas du tout la vérité et où 1 signifie qu'elle la recouvre parfaitement. Plus on obtient une valeur d'mIoU proche de 1 et plus on peut en déduire que la prédiction est bonne. Cependant, c'est une métrique qui a tendance à sous-estimer

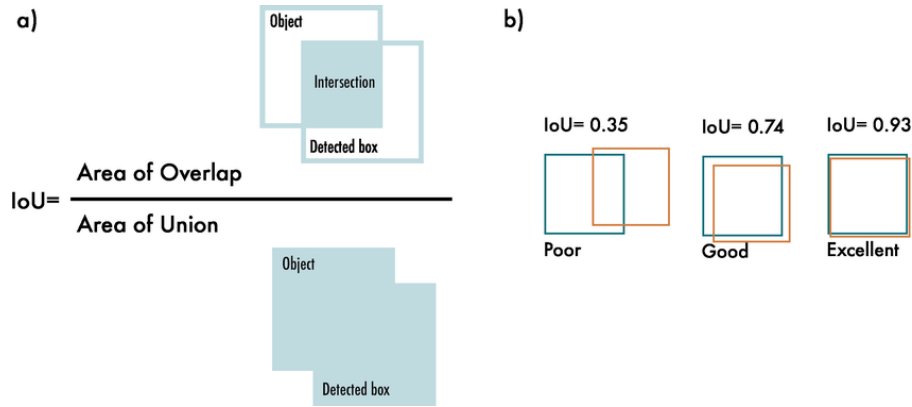


Figure 2.5. Schématisation de l'IoU (figure issue de [18]).

la qualité de la segmentation. En effet, si un objet est mal segmenté mais qu'il recouvre une grande partie de la vérité, l'mIoU aura une valeur convenable alors que la segmentation n'est pas si spécifique au problème, on manquera alors de précision.

Dans la pratique, obtenir une segmentation parfaite, signifiant que l'mIoU vaudra 1, n'est tout simplement pas réaliste. Il est presque impossible que toutes les coordonnées (x, y) de la prédiction recouvrent toutes les coordonnées de la vérité. À travers l'IoU, on cherche donc à récompenser les prédictions qui se superposent bien avec la vérité.

Ainsi il est considéré, dans la pratique, qu'une mIoU supérieure à 0.5 est une mIoU « acceptable » et on considère que si l'mIoU dépasse 0.7, il s'agit d'une « bonne » mIoU (voir [19, 20]). Cette valeur est souvent utilisée comme seuil pour différencier les vrais positifs des faux positifs ou faux négatifs. Par exemple, le jeu de données test *Pascal VOC* [21], qui est utilisé pour la détection d'objet et utilise l'mIoU comme métrique principale, utilise un seuil de 0.5 ce qui signifie que toute détection d'objet ayant une mIoU supérieure à 0.5 est considérée comme une détection positive (vrai positif). Il s'agit d'un consensus commun dans la communauté de la CV, construit sur un compromis entre qualité visuelle et quantitative.

2.3 Création de modèles de segmentation

La particularité de MacTrack2 (voir), la librairie que nous avons développée en utilisant principalement Kartezio, est la facilité à entraîner des modèles. En effet, à l'aide d'un script simple nous pouvons entraîner un modèle de segmentation d'images à partir d'un jeu de données d'images annotées. Nous avons utilisé ImageJ [17] pour annoter nos données. À l'aide de ce logiciel, nous avons pu détourer à la main chacun des macrophages dans les 25 images d'entraînement et 6 images de test que nous avons choisies à travers des films à différents temps post-amputation et conditions (Fig. 2.6)./

Les masques sont données sous la forme de fichiers ROI (Region Of Interest). Ces régions sont représentées dans la Fig. 2.6. Chaque détourage correspond à un

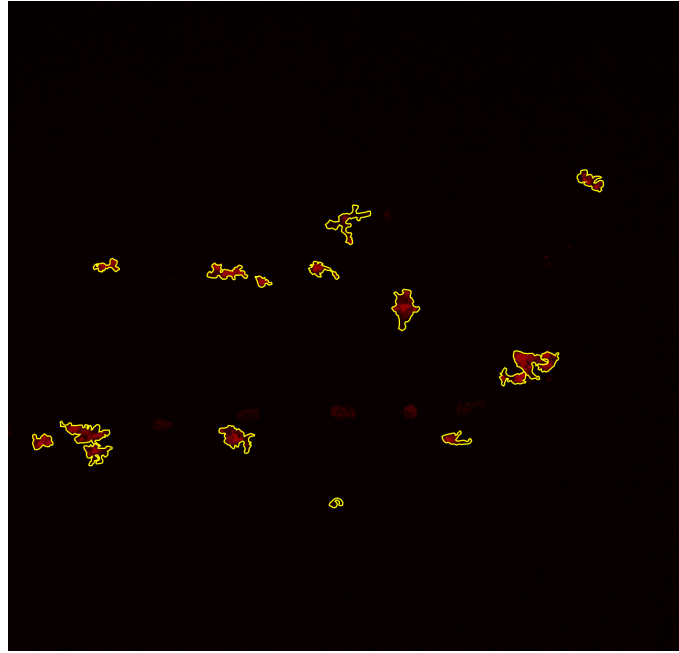


Figure 2.6

fichier ROI et tous les détourages d'une images sont compilé dans un fichier ZIP.

Pour cette trentaine d'image et pour obtenir un détourage précis, cela nous a pris environ 3 jours de travail. Les images que nous avons choisies, ayant différentes conditions (nageoire coupée ou contrôle) et à des temps différents (Fig. 2.7), sont des images pouvant être considérées comme représentatives de la diversité que l'on peut rencontrer lorsque l'on observe des macrophages. En effet, on a souvent des macrophages qui sont proches les uns des autres, on parle de chimio-tactisme, mais aussi des macrophages qui ont des formes variées en fonction de leur état de polarisation. Par exemple, un macrophage dit M1 (pro-inflammatoire) aura tendance à être plutôt rond et compact, on peut l'observer à 3hpa (heures post-amputation), tandis qu'un macrophage M2 (anti-inflammatoire) aura tendance à être plus « filamenteux » et allongé, on peut l'observer a 24hpa (voir figure).

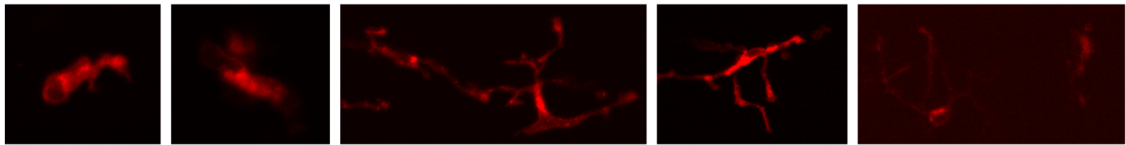


Figure 2.7. Macrophages observés à différents temps post-amputation pour un même poisson.

Les macrophages sont observés, de gauche à droite, à 3, 6, 24, 48 et 72 heures post-amputation (hpa). Il s'agit d'un poisson dont la nageoire caudale a été coupée (ne faisait pas partie du groupe contrôle).

Différence de segmentation entre deux opérateurs

Les annotations d'images sont souvent sujettes à la subjectivité de chaque opérateur détourant les objets d'intérêt. En effet, deux opérateurs, souvent des experts

dans le milieu d'application, peuvent avoir des opinions divergentes ce qui résulte en des annotations différentes. Dans notre cas, comme peut en témoigner la Fig. 2.1, les délimitations des macrophages (dans le canal rouge de l'image) sont parfois floues et peuvent porter à confusion. De plus, certains objets pourtant bien colorés en rouge peuvent être confondus pour des macrophages alors qu'il s'agit de pigments de coloration dus à la fluorescence des protéines.

Nous avons donc voulu comparer les annotations de deux opérateurs, moi-même qui ai annoté l'entièreté des images utilisées pour entraîner les différents modèles, et une membre de l'équipe avec qui j'ai travaillé en collaboration sur ce projet, qui a bien voulu annoter quelques images. La Fig. 2.8 montre en bleu mes annotations et en jaune celles de ma collègue.

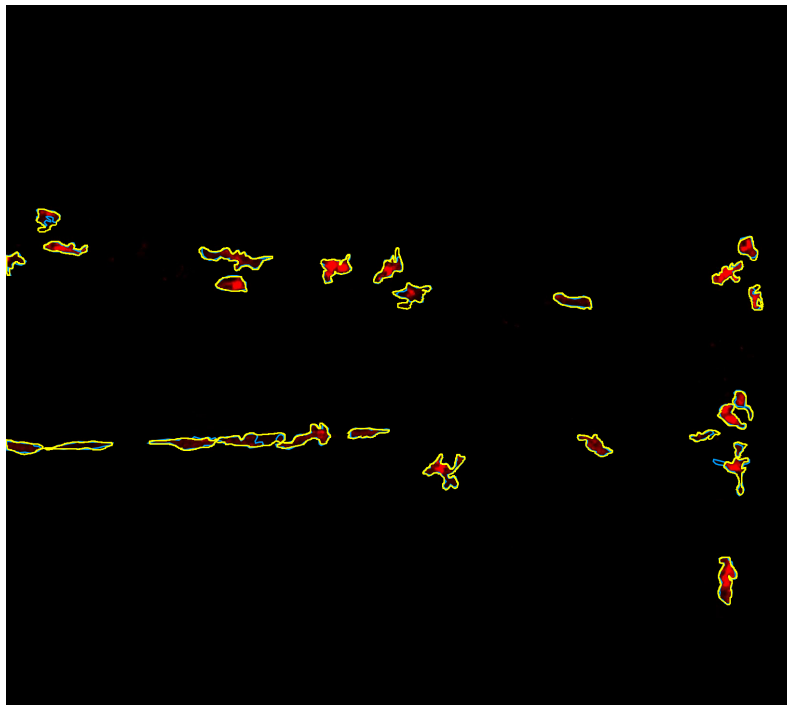


Figure 2.8

Les délimitations des macrophages selon l'opérateur ne sont pas les mêmes, et il y a même des cas où certaines parties du macrophage sont annotées par l'un mais pas par l'autre. Cela se reflète dans la valeur de l'mIoU entre les segmentations des deux opérateurs. En effet, nous obtenons une mIoU de 0.8429. Cela signifie que nous nous rejoignons sur 84,29% des pixels annotés. En général, on considère que la différence de segmentation entre deux experts d'un domaine (que je ne suis pas) est de l'ordre de 15 à 30%.

Nous avons donc quand même une marge d'erreur de 15% entre les deux annotations. De plus, nous avons effectué cette comparaison seulement pour une image. Il aurait fallu le faire pour toutes les images du jeu de données annoté. Cela nous aurait permis de quantifier la différence entre deux opérateurs et de remettre en cause ou non les performances du modèle. En effet, si notre modèle a des performances

similaires à la comparaison entre deux opérateurs humains, cela signifie que le modèle est capable de segmenter les macrophages avec la même variabilité qu'aurait un troisième opérateur humain.

Comparaison des modèles selon le format d'images

Pour des questions de performances et de temps de calculs, nous avons choisi de créer 4 jeux de données issus des mêmes images mais avec des tailles différentes. Dans le premier, l'image était telle quelle, c'est-à-dire de la taille de l'image originale. Dans le second, le modèle *small*, nous avons divisé la taille de l'image par 4, réduisant ainsi le nombre de pixels à traiter par image. Pour le troisième, nous avons retiré, dans les images, les zones noires qui ne sont pas utiles pour la segmentation. Nous avons nommé celui-ci le modèle *cropped*. Dans le dernier cas, nous avons combiné les méthodes du deuxième et du troisième jeu de données, donnant ainsi le modèle *small+cropped*. C'est ce dernier que nous avons gardé pour obtenir les modèles. C'est avec celui-ci que nous avons obtenu les meilleurs résultats dans des temps record. La Tab. 2.1 nous donne une idée des gains entre les différents jeux de données.

Modèle <i>initial</i>		Modèle <i>small</i>		Modèle <i>cropped</i>		Modèle <i>small+cropped</i>	
Train	Test	Train	Test	Train	Test	Train	Test
0.659	0.663	0.639	0.628	0.674	0.706	0.712	0.763
0.647	0.619	0.646	0.581	0.698	0.735	0.687	0.755
0.634	0.639	0.637	0.634	0.682	0.681	0.716	0.727
Temps de calcul							
Train	Test	Train	Test	Train	Test	Train	Test
1h27m	1s	36m30s	0s	49m48s	1s	9m0s	1s
1h28m	2s	5m39s	0s	1h	2s	4m6s	0s
1h54m	3s	7m35s	0s	1h44m	2s	3m48s	0s

Table 2.1. Exemples de performances des modèles entraînés sur les différents jeux de données.

Les valeurs sont les IoU moyens sur le jeu de données d'entraînement et de test. Le modèle dont les performances sont écrites en gras est celui que nous avons conservé pour la suite, et que nous avons fourni en exemple dans le dépôt GitHub du projet. Les différents modèles sont le modèle *initial* avec les images telles quelles, le modèle *small* où le nombre de pixel dans les images est divisé par 4, le modèle *cropped* où les zones noires en haut et en bas des images qui ne sont pas des parties du poisson ont été retirées, et le modèle *small+cropped* qui combine les deux derniers modèles en divisant par 4 le nombre de pixels dans les images du modèle *cropped*. Les temps de calcul sont donnés en heures, minutes et secondes.

2.4 Visualisation de la qualité des modèles

Nous avons mis en place un outil permettant de visualiser la qualité des modèles entraînés. En superposant les prédictions données par le modèle et la vérité sur les images ayant permis d'obtenir ce dernier, nous pouvons visualiser la qualité de la segmentation qu'a le modèle que nous venons de créer. La Fig. 2.9 nous montre, pour

une image venant de l'échantillon d'entraînement et une venant de l'échantillon de test, la superposition des prédictions du meilleur modèle, celui écrit en rouge dans la Tab. 2.1, et de la vérité annotée.

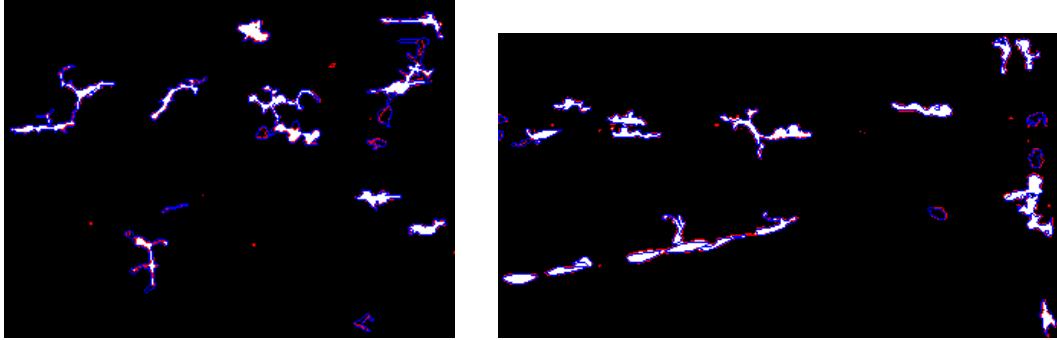


Figure 2.9. Exemples de comparaisons entre la vérité annotée (en bleu) et les prédictions du modèle choisi (en rouge) sur des images issues de l'échantillon d'entraînement (à gauche) et de test (à droite).

Le modèle choisi est celui écrit en rouge dans la Tab. 2.1. Pour l'image d'entraînement, l'IoU est de 0.67 et pour l'image de test, l'IoU est de 0.76. Ce sont toutes les deux des images issues du jeu de données « petit+crop ».

Les prédictions en rouge, obtenues en appliquant le modèle sur chacune des images des deux échantillons, sont parfois très petites en aire, on peut observer des petites taches considérées comme macrophages. Puisque le modèle est entraîné pour repérer des niveaux de gris sur une image, il n'y a pas de distinction réelle sur la question de savoir s'il s'agit bien d'un macrophage ou non.

C'est pour cela que la préparation des images au préalable est une étape primordiale à ne pas négliger. Préparées par l'équipe sur le logiciel ImageJ, les images ont été soigneusement traitées pour faire en sorte de ne pas avoir d'artefacts dans le fond de l'image, et ainsi le fond a une moyenne de gris proche de 0.

2.5 Segmentation automatique d'images avec MacTrack2

Notre librairie, MacTrack2 est capable, à partir des modèles entraînés, de segmenter des images. En effet, la fonction `locate` permet de la faire. À partir d'une image, on effectue la prédiction du modèle sur celle-ci pour obtenir une première segmentation. Nous appliquons un filtre de taille sur les objets détectés afin de laisser de côté les objets trop petits qui ne sont pas des macrophages. Souvent, il s'agit d'artefacts que l'on obtient lors de la prise d'image et ne sont pas visibles à l'œil nu sur l'image d'origine. Nous séparons aussi chacun des objets détectés pour les représenter dans des images différentes. Nous pouvons ainsi observer exclusivement la segmentation de chacun des macrophages. La Fig. 2.10 nous montre les différentes sorties de la fonction `locate`. À gauche, nous retrouvons l'image d'origine, au milieu la segmentation obtenue après application du filtre de taille et à droite la segmentation binaire d'un des macrophages détachés.

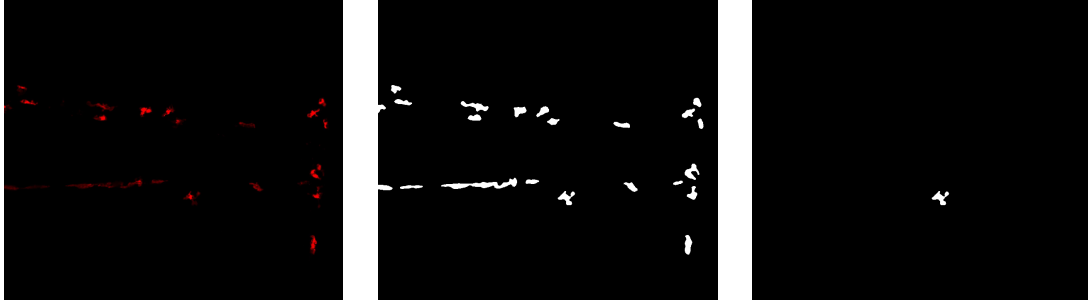


Figure 2.10. Illustrations des différentes sorties de la fonction `locate`

Nous retrouvons à gauche l'image d'origine, au milieu la segmentation complète après application du filtre de taille des objets détectés et à droite la segmentation objet par objet. Ces images proviennent de l'observation de la nageoire caudale d'un poisson à 3h post-amputation. Il s'agit de la deuxième image du film.

Nous avons ainsi obtenu la segmentation de chaque macrophage dans l'image. Cependant, les données que nous avons sont des films, c'est-à-dire des séquences d'images. Il est donc nécessaires d'étendre cette segmentation au domaine temporel. Les dimensions des images sont maintenant $2D+t$. Nous pourrions ensuite appliquer un suivi temporel à la segmentation sémantique obtenue pour chaque image de la vidéo.

Chapitre 3

Suivi temporel des macrophages et fine-tuning d'un modèle de fondation

Les données que nous traitons sont des vidéos qui sont un empilement d'images. Nous allons commencer par étendre la segmentation sémantique obtenue par la librairie MacTrack2 à l'ensemble des images de la vidéo. Nous allons ensuite appliquer un suivi temporel sur chacun d'eux afin de traquer chacun des macrophages de la vidéo. Nous obtiendrons ensuite un tracking des macrophages, nous rapprochant ainsi de la segmentation d'instances.

D'un autre côté, nous allons utiliser un modèle de fondation afin de traiter la vidéo directement et non pas image par image. Ce modèle étant généraliste, il nous sera nécessaire de l'adapter à notre problème en modifiant ses paramètres à travers un *fine-tuning*.

3.1 Tracking à partir de la segmentation d'images

Une fois le modèle généré, visualisé et validé, il est possible de l'appliquer sur des vidéos entières et pas seulement sur une image. Lorsque les vidéos traitées par l'équipe sortent du logiciel, elles sont au format AVI et ont 266 images. Pour plus de détails sur l'obtention des vidéos, voir **Matériel et Méthodes**. Nous commençons donc par convertir les vidéos des canaux rouge et vert et format mp4. Puis, nous découpons cette vidéo en images, les plaçons dans des dossiers spécifiquement rangés pour respecter la structure nécessaire au bon fonctionnement de Kartezio et créons automatiquement des fichiers pour garder cette structure.

En effet, Kartezio s'attend à ce que les images d'entraînement soient présentes dans un dossier « train » qui contient les images (dossier `train_x`) et les fichiers ZIP (dossier `train_y`) de chaque image contenant les annotations des macrophages sous la forme de fichiers ROI. En revanche pour notre vidéo, les images extraites de celle-ci, pour le canal rouge qui nous est utile pour la segmentation des macrophages, sont placées dans le dossier « test », qui contient les images de la vidéo (dossier `test_x`), mais dans le dossier `test_y`, il n'y a pas de fichiers ZIP, car les vidéos que l'on cherche à segmenter automatiquement n'ont pas été annotées à la main. Cependant, pour la structure de Kartezio, il s'attend à ce qu'ils y ait des fichiers ZIP, au même nombre que les images dans le dossier correspondant. C'est

pour cela que nous ajoutons automatiquement des fichiers ZIP vides dans le dossier `test_y`. Nous avons aussi besoin d'un fichier `dataset.csv` (Tab. 3.1) indiquant les images, qu'elles viennent de l'entraînement ou du test qui n'en est pas vraiment un, ainsi que les masques associés même s'ils sont vides, et finalement une colonne « set » qui indique si l'image est une image d'entraînement ou de test (donc issue de la vidéo).

input	label	set
train/train_x/001_frame_crop_small.png	train/train_y/001_masks_crop_small.zip	training
train/train_x/002_frame_crop_small.png	train/train_y/002_masks_crop_small.zip	training
train/train_x/003_frame_crop_small.png	train/train_y/003_masks_crop_small.zip	training

Table 3.1. Extrait d'un fichier `dataset.csv` nécessaire pour la reconnaissance des images et des masques pour Kartezio.

La segmentation de la vidéo peut donc enfin commencer après toute cette étape structurelle fastidieuse mais nécessaire. La première étape de segmentation est réalisée par la fonction `locate`. Elle permet d'obtenir la segmentation, à partir du modèle entraîné, de chaque image dans la vidéo. Il en sort à la fois les images segmenter mais aussi des sous-dossiers contenant les segmentations de chaque macrophage détecté image par image. Autrement dit, nous obtenons un dossier contenant 266 images en noir et blanc de la segmentation des macrophages, mais aussi un dossier contenant des sous-dossiers par macrophages, qui contiennent chacun les images de la segmentation de ce macrophage précis pour le nombre d'images dans lesquelles il a été repéré. La Fig. 2.10 nous montre un exemple des deux sorties obtenues à l'issue de cette première segmentation.

La première étape de segmentation est donc terminée. Puisque nous faisons de la segmentation sémantique, les macrophages proches peuvent être segmentés comme un seul objet. De plus, puisque nous faisons de la microscopie confocale en prenant plusieurs images à différentes profondeurs (*z-stacks*), il est possible qu'un macrophage se cache derrière un autre, on parle d'occlusion.

Defuse pour les objets occlus

Tracking (calcul iou paire par paire de frame pour tous les objets, avec un seuil 0.5 car pas la meme frame et les macro bougent beaucoup donc seuil bas pour un bon compromis, si >0.5 , alors on considère la paire comme étant le meme et on les numérote pareil) plus filters (si ils sont détectés moins d'un certain nombre de frame, on ne les considère pas -> permet de réduire le nombre de macrophages détectés pour coller à la réalité)

Result : ajout du layer de numérotation et des contours, reconstruction de la video final

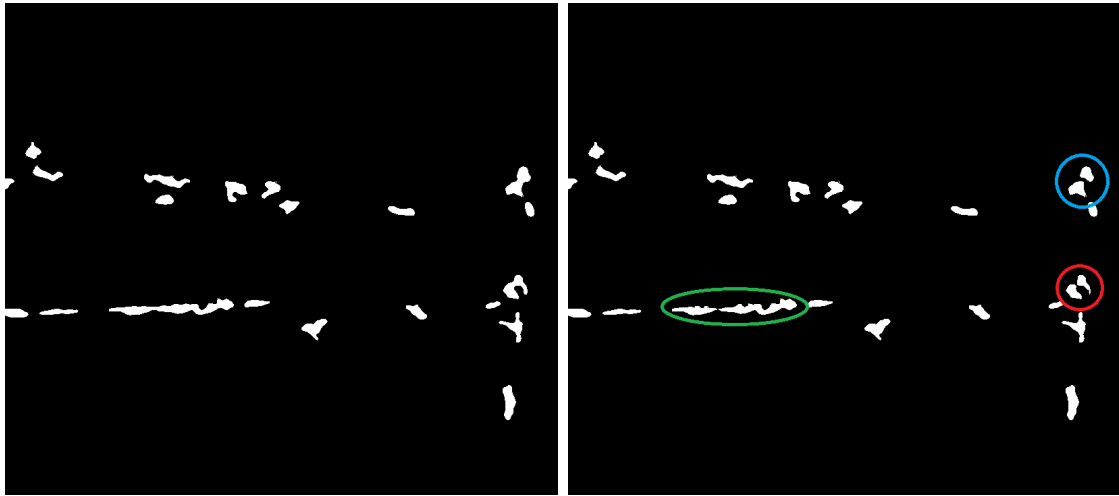


Figure 3.1

3.1.1 Post-traitement

- documentation et site internet, scripts automatiques pour création et exec de modèles avec dataset exemples
- post traitement pour corriger les erreurs de segmentation mettre les images en annexe des différents cas

3.1.2 Analyse des signaux calciques

extraction des données a partir de la segmentation
le faire pour plusieurs vidéos

après l'analyse petite conclusion : on a fait une segmentation framewise de vidéos mais ça reste super rapide et avec des bonnes performances, facilement modifiable et traitable par tout le monde ... pas évident (prise en main (temps, exigence de la structure), accessibilité (docu)) et efficace pour une première étape
on peut maintenant se poser la question de ce qui suit, cad segmenter des vidéos en ayant une mémoire temporelle

nouvelle partie De plus, nous avons une difficulté en plus car jusqu'à présent, nous avons seulement traité de segmentation d'images et non de vidéos. Or, les macrophages étant des cellules immunitaires très mobiles, ils pourraient se regrouper ou se séparer à tout moment de la vidéo. Donc la qualité de la segmentation est primordiale pour pouvoir au mieux retranscrire les comportements réels des macrophages.

figure avec trois images a des frames différentes pour des fusions separations...

3.2 Introduction

SAM-2 (*Segment Anything Model 2*) est un modèle de segmentation d'images et de vidéos développé par Meta AI sorti récemment, en 2024. Il s'agit d'une version

améliorée de la première version du modèle, SAM [22], généralisant le problème de segmentation d'images au domaine des vidéos (*VOS : Video Object Segmentation*). Entraîné sur plusieurs jeux de données dont un premier de 50 000 vidéos, comprenant 600 000 masques (annotations). Les vidéos et masques utilisées pour entraîner un tel modèle proviennent du dataset SA-V [23] créé spécialement pour ce modèle et disponible sur demande. Les annotations ont été à la fois réalisées à la main par des humains mais aussi en étant assisté par la première version de ce modèle, SAM [22] puisqu'elles se font sur des images fixes et qu'il s'agit de la spécialité de la première version (190 000 annotations manuelles et 450 000 annotations automatiques), ce qui accélère le processus d'annotation qui est chronophage et demandeur en personnel. Ces vidéos ont été obtenues par un sous-traitant engagé par Meta FAIR pour récolter des vidéos dans 47 pays différents dans des situations du monde réel. Le second est nommé « *internal dataset* » pour jeu de données de vidéos disponibles sous licence en interne et Meta AI est moins clair sur l'obtention de ces données. Il comprend 62 900 vidéos pour 69 600 annotations. Il a permis d'étendre le jeu de données d'entraînement. Ainsi, SAM-2 est considéré (au même titre que SAM) comme un modèle de fondation (*foundation model*) [24], c'est-à-dire un modèle pré-entraîné sur une grande quantité de données, capable de généraliser à de nombreuses tâches différentes. Il est conçu pour être utilisé comme une base pour des tâches de segmentation en gardant de bonnes performances pour l'ensemble des applications de segmentation d'images et de vidéos.

SAM-2 est capable de segmenter des objets dans des vidéos de manière précise et efficace. L'architecture du modèle (voir Fig. 3.2) est une architecture basée sur les Transformers [25]. Elle comprend un *image encoder* qui est un Vision Transformer hiérarchique appelé Hiera [26], un *prompt encoder* qui prend en compte des clics (positifs ou négatifs), des boîtes ou des masques permettant de définir un objet d'intérêt dans une image et un *mask decoder* qui prend ces entrées et en ressort un masque de segmentation de l'objet sélectionné. Il utilise une mémoire en flux (*streaming memory*). Celle-ci permet de traiter les images d'une vidéo en temps réel, une par une. Ainsi, il peut garder en mémoire, à l'aide de la *memory attention* et de la *memory bank*, les objets et leurs interactions au cours du temps, permettant ainsi de générer des masques plus précis et de les corriger en fonction des images précédentes. La *memory attention* permet de « préparer » l'image en prenant en compte la *memory bank*, qui garde en mémoire les précédentes images, mais aussi les nouveaux points, masques, boîte, que l'utilisateur peut ajouter à chaque image.

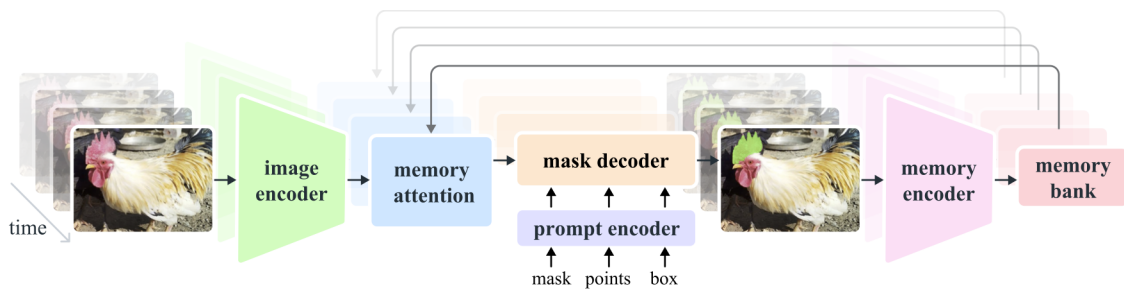


Figure 3.2. Architecture de SAM-2 (figure issue de [27]).

Un des problèmes que nous rencontrons avec SAM-2 est que l'utilisateur doit, à la main, sélectionner des objets d'intérêt dans une ou plusieurs images et si durant la vidéo un nouvel objet apparaît, il ne sera pas détecté tant que l'utilisateur ne l'a pas indiqué. Ainsi, nous allons utiliser la génération automatique de masques mise à disposition pour détecter les macrophages de manière automatique sur des images.

De plus, SAM-2 est un modèle de fondation, conçu pour être utilisé dans différents cas. Cependant, si on veut obtenir de bonnes performances dans un domaine spécifique comme le nôtre, il est nécessaire de faire un *fine-tuning*, c'est-à-dire un ré-entraînement des poids du modèle sur un jeu de données spécifique de notre domaine. Nous allons ainsi fine-tuner SAM-2 pour qu'il puisse mieux détecter les macrophages dans nos vidéos de microscopie.

3.3 Fine-tuning de SAM-2

3.3.1 Définition

Le *fine-tuning* (ou *affinage* en français) est une technique d'apprentissage supervisé qui consiste à modifier les poids d'un modèle pré-entraîné sur une tâche spécifique. Elle est souvent utilisée pour adapter un modèle généraliste à une tâche particulière. Ici, SAM-2 a été entraîné sur des photos qui n'ont rien à voir avec la problématique biologique que nous rencontrons. Par conséquent, afin d'obtenir une bonne segmentation des macrophages, il est nécessaire de modifier les poids du modèle pour obtenir une version spécialisée dans la détection de macrophages. Cela nous évite de devoir entraîner notre propre modèle en partant de rien, ce qui nous demanderait beaucoup trop de données que nous n'avons pas. Mais le nombre de données nécessaires pour obtenir un bon affinage n'est pas négligeable.

Le fine-tuning de SAM-2 agit seulement sur les poids du *prompt encoder* et du *mask encoder*, c'est-à-dire les parties du modèle qui prennent en entrée les points, masques, boîtes, et qui génèrent les masques de segmentation. Nous spécifions donc la prise en charge des entrées données par l'utilisateur, mais pas la partie qui encode l'image ni la partie qui mémorise les images précédente une fois le fine-tuning fini et que l'on cherche à segmenter une vidéo entière.

3.3.2 Jeu de données

Les annotations sont sous la forme d'images où chaque macrophage est détourné à la main sur le logiciel *ImageJ* (et plus particulièrement *Fiji* [28]). Les détournages sont ensuite séparés par label, afin de s'assurer qu'un macrophage ne soit pas confondu avec un autre (Fig. 3.3).

Malgré que le fine-tuning de SAM-2 ne nécessite pas autant de données que l'entraînement d'un modèle de segmentation complet, il est quand même nécessaire de disposer d'un nombre conséquent de données. Ici, nous avons besoin d'images et de leur segmentation manuelle. Or, le temps imparti pour ce stage ne nous permet pas

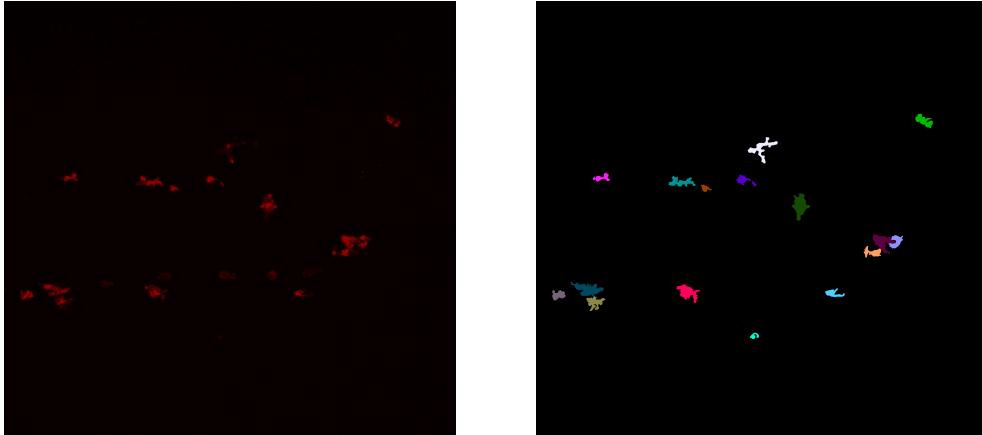


Figure 3.3. Image d'origine (à gauche) comprenant des macrophages et leur détourage (à droite).

de composer un jeu de données complet qui satisferait ces besoins. Par conséquent, nous avons utilisé le programme MacTrack2 qui nous a permis de segmenter une vidéo de macrophages. Ainsi, nous disposons de 297 images et masques de macrophages (dont 266 faisant partie d'une seule vidéo) pour ré-entraîner les poids du modèle.

3.3.3 Hyper-paramètres

Les hyper-paramètres, dans un modèle, constituent les paramètres non appris par le modèle durant l'entraînement (ou le ré-entraînement). Ils sont fixés à l'avance et influencent son comportement et par extension ses performances. Les hyper-paramètres du pré-entraînement et de l'entraînement complet de SAM-2 sont disponibles dans leur papier [27] page 19.

Pour le fine-tuning, les sous-modules affectés sont le *mask decoder* et le *prompt encoder*. Nous utilisons l'optimiseur AdamW [29] avec un taux d'apprentissage (*learning rate*) de 0.0001 et une dégradation des poids (*weight decay*) de 0.00001, utile pour la régularisation L2. Le nombre total d'itérations est de 20 000, avec un nombre de pas pour l'accumulation des gradients (*accumulation step*) avant leur mise à jour égal à 4 steps. Le *scheduler* permet d'ajuster le *learning rate* durant l'entraînement pour améliorer la convergence du modèle. Ici nous utilisons un *stepLR scheduler*, ce qui signifie que tous les k steps, le *learning rate* est multiplié par un facteur γ . Nous avons $k = 500$ et $\gamma = 0.1$. Ainsi, le *learning rate* est réduit de 90% tous les 500 pas.

Contrairement à ce que l'on peut retrouver habituellement dans l'entraînement de modèles, la taille du batch est égale à une seule image. En effet, à chaque pas ...

3.3.4 Fonctions de perte

Pour entraîner un modèle, il est nécessaire de définir une fonction de perte $\mathcal{L}$ à minimiser. Elle est utilisée pour quantifier la qualité des prédictions du modèle par

rapport à la « vérité » (ground truth). Dans le cas du fine-tuning de SAM-2, nous utilisons une combinaison de deux fonctions de perte : la *binary cross-entropy loss* qui sert de perte pour la segmentation et la *score loss* qui est utilisée en complément pour évaluer la qualité des prédictions des masques à partir de la métrique IoU (*Intersection over Union*). L'IoU est une métrique grandement utilisée dans les problèmes de segmentation d'images. Elle est définie dans la partie 3.1 et peut être illustrée par la Fig. 2.5.

En moyennant sur tous les objets d'une image, on obtient la mIoU (IoU moyenne), comme obtenue par 2.1 dans la partie 2.2

Ainsi, nous pouvons définir la *score loss* comme suit :

$$\mathcal{L}_{\text{score}} = \frac{1}{K} \sum_{k=1}^K |s_k - \text{IoU}(M_k, G_k)|$$

avec K le nombre total de masques (donc de macrophages ou d'objets) dans l'image et s_k le score prédit pour l'objet k à partir de M_k .

Nous comparons donc le score prédit par le modèle pour chaque masque à la véritable qualité de chacun d'eux (l'IoU). Puisque'on cherche à minimiser cette perte, on aimerait que pour tout k , $s_k \simeq \text{IoU}(M_k, G_k)$ pour que la perte soit proche de 0.

Si $K = 0$, alors cela signifie qu'il n'y a pas de masques dans l'image, donc pas de macrophages. Un tel cas peut être rencontré, mais nous avons décidé de ne pas l'inclure dans le fine-tuning, car il n'apporte pas d'information utile concernant la segmentation des macrophages.

D'un autre côté, la *binary cross-entropy loss* $\mathcal{L}_{BCE}$ est une fonction de perte généralement utilisée dans des problèmes de classification binaire, et notamment dans les problèmes de segmentation d'images. Elle est utilisée pour quantifier la différence entre les prédictions du modèle et les véritables annotations (masques de vérité). Elle est définie comme suit :

$$\mathcal{L}_{BCE} = -\frac{1}{K \times H \times W} \sum_{k=1}^K \sum_{i=1}^H \sum_{j=1}^W y_{k,i,j} \log \hat{y}_{k,i,j} + (1 - y_{k,i,j}) \log(1 - \hat{y}_{k,i,j})$$

avec :

- $y_{k,i,j} \in \{0, 1\}$ la valeur du pixel (i, j) du masque de vérité pour l'objet k . On peut notamment remarquer que la matrice $G_k = (y_{k,i,j})_{i,j}$ est une matrice binaire (composée de 0 et de 1) de même dimension que les dimensions de l'image ($H \times W$).
- $\hat{y}_{k,i,j} \in [0, 1]$ est la probabilité que le pixel (i, j) appartienne au masque de l'objet k . Elle est obtenue par la transformation sigmoïde appliquée à la prédiction des masques du modèle, qui sont des logits générés par le *mask decoder* (voir Tab. 3.2). On a donc :

$$\hat{y}_{k,i,j} = \sigma(z_{k,i,j}) = \frac{1}{1 + \exp(-z_{k,i,j})}$$

avec $z_{k,i,j}$ le logit du pixel (i, j) du masque prédit pour l'objet k .

Logit ($z_{k,i,j}$)	Interprétation	Sigmoid ($\hat{y}_{k,i,j}$)
+10	Très sûr qu'il s'agit d'un objet	≈ 1
+1	Assez confiant qu'il s'agit d'un objet	≈ 0.75
0	Incertitude totale	0.5
-1	Assez confiant qu'il s'agit du fond de l'image	≈ 0.25
-10	Très sûr qu'il s'agit du fond de l'image	≈ 0

Table 3.2. Différents exemples des valeurs que peuvent prendre les sorties du *mask decoder* (logits), leur interprétation et la valeur de la probabilité associée (par transformation sigmoïde).

- H et W la hauteur et la largeur de l'image.
- K le nombre total de masques (donc de macrophages ou d'objets) dans l'image.

Interprétations fonction de perte ...ressemble a kullbackleibler mais différences ...

Finalement, la fonction de perte totale est une combinaison de ces deux dernières :

$$\mathcal{L} = \mathcal{L}_{BCE} + \lambda \mathcal{L}_{\text{score}}$$

avec λ un hyper-paramètre qui permet de pondérer l'importance de la *score loss* par rapport à la *binary cross-entropy loss*. Dans notre cas, nous avons fixé λ à 0.05.

3.3.5 Optimisation

Une fois la perte calculée, elle est divisée par le nombre de pas d'accumulation des gradients, soit 4 dans notre cas, comme défini dans la partie 3.3.3 introduisant les hyper-paramètres. Cela nous permet d'éviter de multiplier le gradient total par ce facteur. C'est utilisé lorsque l'on accumule des gradients sur plusieurs images (ici 4 fois une image) pour que la somme finale corresponde en réalité à une moyenne sur ce batch étendu. Ainsi, nous simulons un batch de taille 4 alors qu'en réalité nous n'avons qu'une seule image par step.

Ensuite, les gradients sont calculés en utilisant la méthode *backward* du package PyTorch [30], à partir de la perte totale $\mathcal{L}$ divisée par cet *accumulation step*. Puis, tous ces 4 steps, nous mettons à jour l'optimiseur AdamW. Le nombre de pas avant la mise à jour du *learning rate* est de 500. Ainsi, le *learning rate* est réduit de 90% tous les 500 pas. Ensuite, les gradients sont remis à zéro pour le prochain pas d'accumulation.

Pour finir, nous calculons l'IoU moyenne au pas ℓ comme étant :

$$\text{mIoU}_\ell = \begin{cases} 0.9 \cdot \text{mIoU}_{\ell-1} + 0.1 \cdot \frac{1}{K} \sum_{k=1}^K \text{IoU}(M_k, G_k) & \text{si } k > 0, \\ 0 & \text{sinon.} \end{cases}$$

3.3.6 Résultats et comparaisons

graphes miou et loss comp avant apres fine tune, performances et visuel

3.4 Propagation dans une vidéo

Chapitre 4

Conclusion

Disponibilité du code

Le code de MacTrack2 est disponible, ainsi que deux scripts détaillés pour la création de modèle et l'analyse d'une vidéo, avec un jeu de données d'exemple pour chacun des deux scripts, à l'adresse suivante : <https://github.com/guibouland/MacTrack2>.

Un site internet est disponible sur ce dépôt GitHub, regroupant toute la documentation et des instructions détaillées pour les utilisateurs plus novices, notamment les biologistes traitant la même problématique, qui sont la cible principale de ce projet. Il servira aussi aux futurs membres du laboratoire LPHI qui souhaitent utiliser MacTrack2 pour leurs propres projets.

Chapitre 5

Matériel et Méthodes

tests tests[5, 22, 27, 29]

Acquisition des images

microscope . . .norma merci

Format des vidéos

reconstitution sur imagej... par norma images toutes les 9sec donc sur 40 min de prise de vidéo on a que 266 images

Fine-tuning de SAM-2

L'optimiseur AdamW a été utilisé avec comme hyper-paramètres un taux d'apprentissage (*learning rate*) de 0.0001, une dégradation de pondération (*weight decay*) de 0.0001. L'entraînement a été effectué avec 20000 itérations. Nous avons utilisé un GPU Nvidia A10 Tensor Core avec 24 Go de mémoire.

Annexe A

Annexes

A.1 Superposition des deux canaux

A.2 Liste des fonctions de traitement d'images disponibles dans Kartezio

Numéro	Fonction	Description
1	max	
2	min	
3	mean	
4	add	
5	subtract	
6	bitwise_not	
7	bitwise_or	
8	bitwise_and	
9	bitwise_and_mask	
10	bitwise_xor	
11	sqrt	
12	pow2	
13	exp	
14	log	
15	median_blur	
16	gaussian_blur	
17	laplacian	
18	sobel	
19	robert_cross	
20	canny	
21	sharpen	
22	gabor	
23	abs_diff	
24	abs_diff2	
25	fluo_tophat	

Numéro	Fonction	Description
26	rel_diff	
27	erode	
28	dilate	
29	open	
30	close	
31	morph_gradient	
32	morph_tophat	
33	morph_blackhat	
34	fill_holes	
35	remove_small_objects	
36	remove_small_holes	
37	threshold	
38	threshold_at_1	
39	distance_transform	
40	distance_transform_and_thresh	
41	inrange_bin	
42	inrange	

Table A.1. Fonctions et descriptions

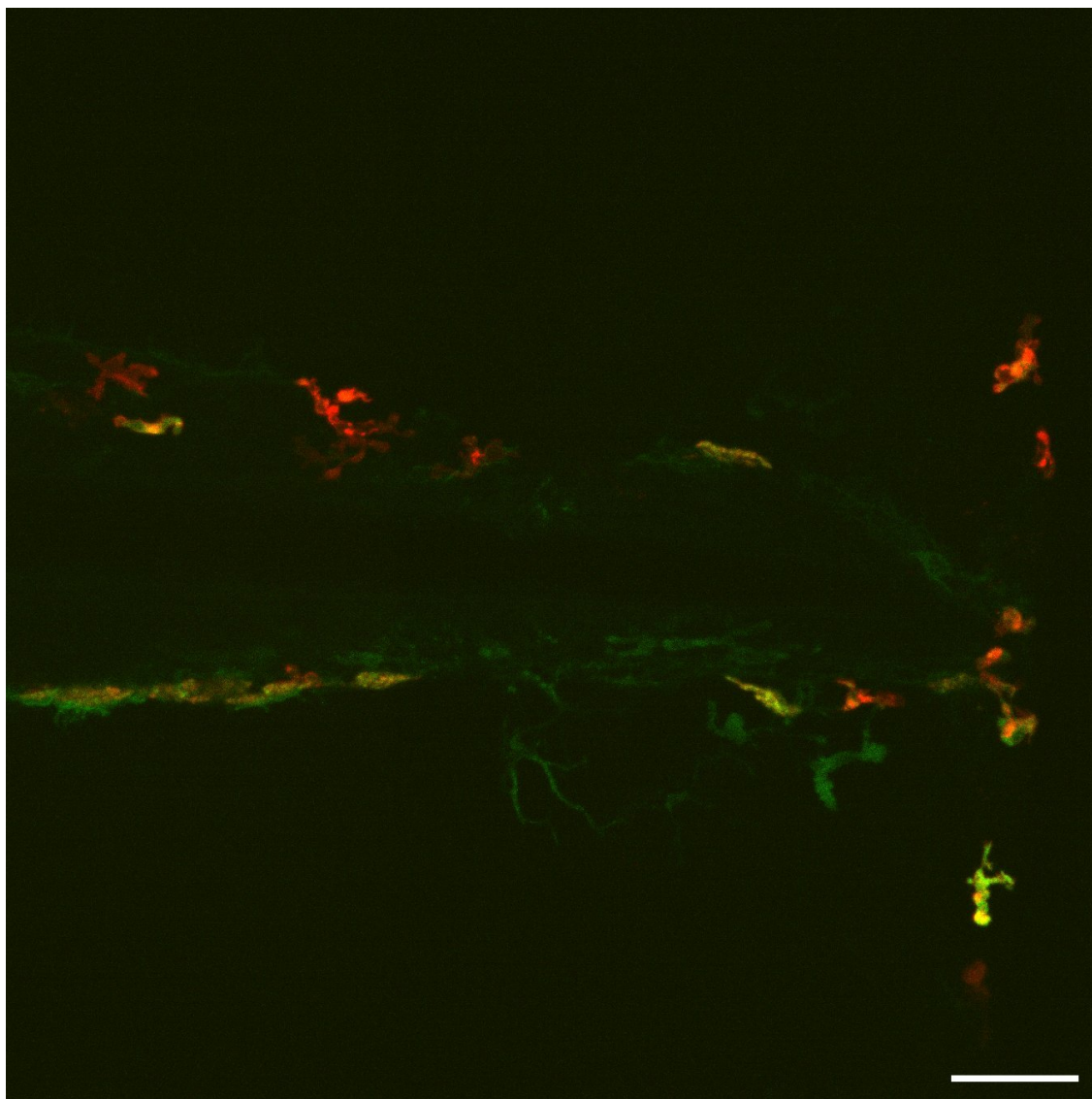


Figure A.1. Superposition de deux canaux de couleurs.

Bibliographie

- [1] Qing JIANG et al. *T-Rex2 : Towards Generic Object Detection via Text-Visual Prompt Synergy*. 2024. arXiv : [2403.14610 \[cs.CV\]](#). URL : <https://arxiv.org/abs/2403.14610>.
- [2] Tianhe REN et al. *Grounded SAM : Assembling Open-World Models for Diverse Visual Tasks*. 2024. arXiv : [2401.14159 \[cs.CV\]](#).
- [3] Shilong LIU et al. « Grounding dino : Marrying dino with grounded pre-training for open-set object detection ». In : *arXiv preprint arXiv :2303.05499* (2023).
- [4] Tianhe REN et al. *Grounding DINO 1.5 : Advance the "Edge" of Open-Set Object Detection*. 2024. arXiv : [2405.10300 \[cs.CV\]](#).
- [5] Kévin CORTACERO et al. « Evolutionary design of explainable algorithms for biomedical image segmentation ». In : *Nature Communications* 14.1 (2023), p. 7112.
- [6] Carsen STRINGER et al. « Cellpose : a generalist algorithm for cellular segmentation ». In : *Nature methods* 18.1 (2021), p. 100-106.
- [7] Marius PACHITARIU et Carsen STRINGER. « Cellpose 2.0 : how to train your own model ». In : *Nature methods* 19.12 (2022), p. 1634-1641.
- [8] Carsen STRINGER et Marius PACHITARIU. « Cellpose3 : one-click image restoration for improved cellular segmentation ». In : *Nature Methods* (2025), p. 1-8.
- [9] Kaiming HE et al. « Mask r-cnn ». In : *Proceedings of the IEEE international conference on computer vision*. 2017, p. 2961-2969.
- [10] Uwe SCHMIDT et al. « Cell detection with star-convex polygons ». In : *Medical image computing and computer assisted intervention—MICCAI 2018 : 21st international conference, Granada, Spain, September 16-20, 2018, proceedings, part II* 11. Springer. 2018, p. 265-273.
- [11] Julian MILLER et Andrew TURNER. « Cartesian Genetic Programming ». In : *Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation*. GECCO Companion '15. Madrid, Spain : Association for Computing Machinery, 2015, p. 179-198. ISBN : 9781450334884. DOI : [10.1145/2739482.2756571](#).
- [12] Julian F MILLER, Peter THOMSON, Terence FOGARTY et al. « Designing electronic circuits using evolutionary algorithms. arithmetic circuits : A case study ». In : *Genetic algorithms and evolution strategies in engineering and computer science* 10 (1997).
- [13] M-J WILLIS et al. « Genetic programming : An introduction and survey of applications ». In : *Second international conference on genetic algorithms in engineering systems : innovations and applications*. IET. 1997, p. 314-319.

- [14] Johannes TEXTOR. *Dagitty*. <https://github.com/jtextor/dagitty>. 2010.
- [15] OpenCV TEAM. *OpenCV : Open source Computer Vision Library*. <https://opencv.org/>.
- [16] OpenCV TEAM. *OpenCV : Open source Computer Vision Library - Python*. <https://github.com/opencv/opencv-python>.
- [17] Caroline A SCHNEIDER, Wayne S RASBAND et Kevin W ELICEIRI. « NIH Image to ImageJ : 25 years of image analysis ». In : *Nature methods* 9.7 (2012), p. 671-675.
- [18] Juan TERVEN et Diana-Margarita CORDOVA-ESPARZA. *A Comprehensive Review of YOLO : From YOLOv1 to YOLOv8 and Beyond*. Avr. 2023. DOI : [10.48550/arXiv.2304.00501](https://doi.org/10.48550/arXiv.2304.00501).
- [19] Gaundez BOESCH. *What is Intersection over Union (IoU) ?* <https://viso.ai/computer-vision/intersection-over-union-iou/>. 2024.
- [20] Zhaowei CAI et Nuno VASCONCELOS. *Cascade R-CNN : Delving into High Quality Object Detection*. 2017. arXiv : [1712.00726](https://arxiv.org/abs/1712.00726) [cs.CV]. URL : <https://arxiv.org/abs/1712.00726>.
- [21] Mark EVERINGHAM et al. « The Pascal Visual Object Classes (VOC) challenge ». In : *International Journal of Computer Vision* 88 (juin 2010), p. 303-338. DOI : [10.1007/s11263-009-0275-4](https://doi.org/10.1007/s11263-009-0275-4).
- [22] Alexander KIRILLOV et al. « Segment Anything ». In : (2023). arXiv : [2304.02643](https://arxiv.org/abs/2304.02643) [cs.CV]. URL : <https://arxiv.org/abs/2304.02643>.
- [23] Meta FAIR. *SA-V Dataset*. <https://ai.meta.com/datasets/segment-anything-video/>. 2024.
- [24] Rishi BOMMASANI et al. *On the Opportunities and Risks of Foundation Models*. 2022. arXiv : [2108.07258](https://arxiv.org/abs/2108.07258) [cs.LG]. URL : <https://arxiv.org/abs/2108.07258>.
- [25] Ashish VASWANI et al. *Attention Is All You Need*. 2023. arXiv : [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. URL : <https://arxiv.org/abs/1706.03762>.
- [26] Chaitanya RYALI et al. *Hiera : A Hierarchical Vision Transformer without the Bells-and-Whistles*. 2023. arXiv : [2306.00989](https://arxiv.org/abs/2306.00989) [cs.CV]. URL : <https://arxiv.org/abs/2306.00989>.
- [27] Nikhila RAVI et al. « SAM 2 : Segment Anything in Images and Videos ». In : *arXiv preprint arXiv :2408.00714* (2024). URL : <https://arxiv.org/abs/2408.00714>.
- [28] Johannes SCHINDELIN et al. « Fiji : an open-source platform for biological-image analysis ». In : *Nature Methods* 9.7 (2012), p. 676-682. DOI : [10.1038/nmeth.2019](https://doi.org/10.1038/nmeth.2019).
- [29] Ilya LOSHCHILOV et Frank HUTTER. « Decoupled Weight Decay Regularization ». In : (2019). arXiv : [1711.05101](https://arxiv.org/abs/1711.05101) [cs.LG]. URL : <https://arxiv.org/abs/1711.05101>.
- [30] Adam PASZKE et al. « PyTorch : An Imperative Style, High-Performance Deep Learning Library ». In : (déc. 2019). DOI : [10.48550/arXiv.1912.01703](https://doi.org/10.48550/arXiv.1912.01703).